

開始使用 Google MCP 伺服器

來源：<https://codelabs.developers.google.com/getting-started-google-mcp-servers?hl=zh-tw>

步驟數：10

本程式碼研究室會逐步引導您安裝 Google Antigravity (首重代理程式的開發平台)，並體驗相關功能。

目錄

1. 簡介
2. 事前準備
3. 基礎知識：連線至第一個 MCP 伺服器
4. 歷程 A：大腦 (Developer Knowledge MCP)
5. 旅程 B：分類 (自動疑難排解)
6. 歷程 C：資料 (Firestore MCP)
7. 歷程 D：智慧 (BigQuery 和 Google 地圖)
8. 強化：生產環境安全和 IAM
9. 清除所用資源
10. 結論

1. 簡介

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

歡迎！在本程式碼研究室中，您將瞭解如何使用 **Google 管理的 Model Context Protocol (MCP) 伺服器**，大幅提升 AI 代理的效能。

Model Context Protocol (MCP) 是一項開放原始碼標準，可讓 AI 模型安全有效地連結至外部資料來源和工具。雖然大多數 MCP 實作項目都會在本機執行，但 Google 提供代管的遠端 **MCP 伺服器**。這些是全代管的企業級端點，可讓代理程式直接與 Google Cloud 雲端基礎架構互動，您不必管理任何伺服器端程式碼或容器。

「受管理」優勢

與使用標準輸入/輸出 (stdio) 的本機 MCP 伺服器不同，Google 管理的伺服器會使用可串流的 **HTTP**。此架構提供以下功能：

- **零基礎架構**：無須佈建或擴充伺服器。
- **設計安全防護機制**：與 Google Cloud IAM 和稽核記錄原生整合。
- **無狀態擴充**：透過標準負載平衡器和 Proxy 順暢互動。

課程內容

- 如何啟用及驗證代管的 MCP 伺服器。
- 如何使用 **Cloud Logging MCP 伺服器** 做為基礎基準。
- 如何自動調度管理多個 MCP 伺服器 (Developer Knowledge、Firestore 等)，建構自主工作流程。

軟硬體需求

- 已啟用計費功能的 Google Cloud 專案。
- 熟悉 Google Cloud 控制台和 `gcloud` CLI。
- **Google Cloud Shell** (已預先安裝 Gemini CLI)。

本程式碼研究室適合各種程度的使用者和開發人員 (包括初學者) 參加。

報表問題

在逐步完成程式碼研究室和使用 Antigravity 時，您可能會遇到問題。

如要回報程式碼研究室相關問題 (錯字、錯誤的操作說明)，請點選本程式碼研究室左下角的 `Report a mistake` 按鈕開啟錯誤回報單：

 Report a mistake

步驟 2 · 約 5 分鐘

2. 事前準備

在本步驟中，您將準備 Google Cloud 環境。我們將在 **Google Cloud Shell** 中執行所有工作，這個工具提供預先設定好的永久終端機。

啟用 Cloud Shell

1. 前往 [Google Cloud 控制台](#)。
2. 按一下右上角標題中的「啟用 Cloud Shell」圖示。
3. 終端機工作階段啟動後，系統會要求您授權提示。

設定專案 ID

確認 Cloud Shell 指向正確的專案：

```
# Set your active project
gcloud config set project YOUR_PROJECT_ID

# Verify the setting
gcloud config list project
```

啟用基礎模型 API

代管 MCP 伺服器需要啟用基礎產品 API 和 MCP 介面。執行下列指令，啟用 **Cloud Logging** 後端 (本實驗室的基準)：

```
# Enable the Cloud Logging API and its MCP interface
gcloud services enable logging.googleapis.com
gcloud beta services mcp enable logging.googleapis.com
```

注意：受管理 MCP 服務目前為 **Beta 版**。您必須使用 gcloud Beta 版元件才能啟用這些功能。

設定應用程式預設憑證 (ADC)

Gemini CLI 會使用您的使用者身分與 MCP 伺服器通訊。授予代理人權限，允許對方代您執行動作：

```
gcloud auth application-default login
```

按照終端機中的網址登入，然後將授權碼貼回 Cloud Shell。

指派基礎 IAM 角色

代管 MCP 伺服器採用**雙層安全模式**。你必須開啟兩個特定「閘道」：

1. **閘道 1 (MCP 存取權)**：可呼叫通訊協定的角色。
2. **第 2 道閘 (服務存取)**：可查看資料的角色 (例如查看記錄)。

執行下列指令，將必要存取權授予自己：

```
export PROJECT_ID=$(gcloud config get-value project)
export USER_EMAIL=$(gcloud config get-value account)

# Gate 1: Permission to use the MCP protocol
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="user:$USER_EMAIL" \
  --role="roles/mcp.toolUser"

# Gate 2: Permission to view the actual logs
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="user:$USER_EMAIL" \
  --role="roles/logging.viewer"
```

步驟 3 · 約 15 分鐘

3. 基礎知識：連線至第一個 MCP 伺服器

在本步驟中，您會將 AI 代理程式 (Gemini CLI) 連結至 **Google Cloud Logging MCP 伺服器**。這是我們的「基礎」，因為代理程式可以即時查看專案內發生的情況。

工作 1：設定記錄 MCP 伺服器

Gemini CLI 會使用 settings.json 檔案管理連線。您需要編輯這個檔案 (位於 `~/.gemini` 資料夾中)，在 mcpServers 區塊中加入下列程式碼片段。將 `YOUR_PROJECT_ID` 替換為實際專案 ID：

```
"logging-mcp": {
  "httpUrl": "https://logging.googleapis.com/mcp",
  "authProviderType": "google_credentials",
  "oauth": {
    "scopes": [
      "https://www.googleapis.com/auth/logging.read"
    ]
  },
  "timeout": 30000,
  "headers": {
    "x-goog-user-project": "YOUR_PROJECT_ID"
  }
}
```

注意：管理型 MCP 伺服器必須使用 `x-goog-user-project` 標頭，才能確保 API 用量和帳單正確歸給專案。

工作 2：模擬專案活動 (建立記錄)

如果專案是新建立或處於閒置狀態，可能不會有任何「有趣的」近期記錄。我們使用 `gcloud` CLI 插入幾個自訂項目，讓代理程式有東西可尋找。

請逐一執行下列指令，模擬一連串事件：

```
# 1. Simulate a standard system start
gcloud logging write mcp-test-log "System boot sequence initiated" --severity=INFO
# 2. Simulate a warning about resource limits
gcloud logging write mcp-test-log "High memory pressure detected in zone us-central1-a" --
severity=WARNING
# 3. Simulate a critical authentication failure
gcloud logging write mcp-test-log "ERROR: Failed to connect to Cloud SQL. Permission Denied."
--severity=ERROR
```

工作 3：在 Gemini CLI 中驗證工具

開始對話前，請先確認代理程式可以「看到」Logging 伺服器公開的工具。啟動 Gemini CLI：

```
gemini
```

進入 Gemini CLI 提示 (`>`) 後，執行 `list` 指令：

```
/mcp list
```

驗證檢查點：您應該會看到 `logging-mcp` 列為「就緒」，並提供約 **6 項工具**，包括 `list_log_entries`。

工作 4：第一個即時基礎架構提示

現在，請代理程式找出我們剛建立的記錄。由於您先前已授予 `roles/logging.viewer` 角色，因此代理程式現在可以「聯絡」並讀取專案狀態。

在 **Gemini CLI** 中輸入下列提示：

```
Show me the 3 most recent log entries from the log named 'mcp-test-log'. What is the highest severity issue you see?
```

觀察服務專員：

1. 代理程式可能會提示您輸入 Google Cloud 專案 ID。請提供該資訊。
 2. 並判斷需要使用 `list_log_entries` 工具。
 3. 系統會要求您授權執行這項工具。選取「1. Yes, allow once」按下 Enter 鍵來確認操作。
 4. 這項工具會剖析 JSON 回應，並說明我們模擬的 **Cloud SQL** 權限遭拒錯誤。
-

4. 歷程 A：大腦 (Developer Knowledge MCP)

在這趟旅程中，您會將代理連結至 **Google Developer Knowledge MCP 伺服器**，為代理提供「大腦」。

AI 代理的最大風險之一是產生幻覺，也就是自信地提供過時的 CLI 指令或已淘汰的 API 參數。為解決這個問題，MCP 伺服器會根據 Google 官方即時開發人員文件集 (涵蓋 Google Cloud、Firebase、Android 等) 建立代理程式。

工作 1：啟用知識服務

與基礎步驟相同，我們必須啟用後端 API 和 MCP 服務端點。

```
# 1. Enable the Developer Knowledge API
gcloud services enable developerknowledge.googleapis.com

# 2. Enable the MCP Server interface
gcloud beta services mcp enable developerknowledge.googleapis.com
```

工作 2：佈建受限制的 API 金鑰

Developer Knowledge MCP 使用 **API 金鑰** 進行驗證。為確保安全，我們會建立金鑰並加以限制，*只能用於這個特定 API*。

1. 執行下列指令碼來建立及擷取金鑰：

```
# Create the restricted API key
gcloud alpha services api-keys create \
  --display-name="MCP-Knowledge-Key" \
  --api-target service=developerknowledge.googleapis.com

# Wait a few seconds for the key to propagate, then fetch the string
gcloud alpha services api-keys get-key-string \
  $(gcloud alpha services api-keys list \
    --filter="displayName='MCP-Knowledge-Key'" \
    --format="value(name)") \
    --format="value(keyString)"
```

2. 複製第二個指令傳回的長字串。這是你的 `YOUR_API_KEY`。

工作 3：設定 Gemini CLI

現在，請向代理註冊 Knowledge MCP 伺服器。這樣一來，如果遇到無法完全確定的技術問題，代理程式就能搜尋官方文件。

在 `~/.gemini/settings.json` 檔案的 `mcpServers` 區段中新增下列程式碼片段，並將 `YOUR_API_KEY` 替換成您剛複製的字串：

```
"developer-knowledge-mcp": {
  "httpUrl": "https://developerknowledge.googleapis.com/mcp",
  "headers": {
    "X-Goog-API-Key": "YOUR_API_KEY"
  }
}
```

工作 4：反幻覺測試

現在來驗證代理程式是否「研究」而非「猜測」。

啟動 Gemini CLI：

```
gemini
```

確認伺服器已準備就緒：輸入 `/mcp list`。您應該會看到 `google-developer-knowledge`，其中包含 2 項工具（`search_documents` 和 `get_document`）。

提示：要求代理程式尋找特定新式指令。

```
I want to create a Google Cloud Storage bucket using the modern gcloud storage command.
Search the official documentation for the exact syntax and show me an example for a bucket in
the 'us-centrall' region.
```

報表重點：

- Gemini 會要求使用 `search_documents` 的權限。
 - 接著，系統可能會呼叫 `get_document` 來朗讀找到的特定頁面。
 - 最終答案應包含 `gcloud storage buckets create ...` 指令，直接引用自說明文件。
-

5. 旅程 B：分類 (自動疑難排解)

必要條件：您必須完成學習路徑 **A：大腦**，代理程式才能研究修正方式。

在這項旅程中，您將結合代理程式的「眼睛」(Cloud Logging MCP) 和「大腦」(Developer Knowledge MCP)，建構「自主疑難排解迴圈」。

您不必手動將錯誤代碼複製到搜尋引擎，只要提供單一提示，代理程式就會掃描專案中的錯誤、研究官方解決方案，並產生可執行的修正報告。

工作 1：在 GCP 中模擬「不順利的一天」

若要瞭解自主疑難排解的強大功能，我們需要一組實際的失敗案例。我們會使用 Python 指令碼，將各種基礎架構障礙 (從權限遭拒錯誤到配額問題) 直接注入記錄中。

1. 在 Cloud Shell 中建立所選資料夾，然後前往該資料夾。
2. 建立名為 `simulate_errors.py` 的檔案：

```
nano simulate_errors.py
```

3. 將下列程式碼貼入編輯器：

```
import argparse
from google.cloud import logging

def simulate_errors(project_id):
    client = logging.Client(project=project_id)
    logger = client.logger("mcp-scenario-logger")

    print(f"Simulating realistic errors for project: {project_id}...")

    # 1. GCS Permission Error
    logger.log_text("ERROR: GCS Upload failed for 'gs://my-app-bucket/data.json'. Status: 403 Forbidden. Missing 'storage.objects.create' for service account.", severity="ERROR")

    # 2. Cloud Run Startup Error
    logger.log_text("ERROR: Cloud Run service 'api-gateway' failed to start. Container failed to listen on port 8080. Check 'Cloud Run container startup requirements'.", severity="ERROR")

    # 3. Secret Manager Access Error
    logger.log_text("ERROR: Access denied to secret 'API_KEY'. The identity lacks 'secretmanager.versions.access'.", severity="ERROR")

    print("Log entries written to 'mcp-scenario-logger'.")

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--project", required=True)
    args = parser.parse_args()
    simulate_errors(args.project)
```

4. 依序按下 Ctrl+O 鍵、Enter 鍵和 Ctrl+X 鍵，即可儲存並結束。
5. 安裝 Google Cloud Logging 程式庫並執行指令碼：

```
python -m venv mcp_env
source mcp_env/bin/activate
pip install google-cloud-logging
python simulate_errors.py --project $(gcloud config get-value project)
```

工作 2：執行自主迴圈

現在，我們要傳送複雜的提示，指示 Gemini 同時協調兩個 MCP 伺服器。

啟動 Gemini CLI：

```
gemini
```

在代理中輸入這項「主提示」：

```
I need to troubleshoot recent issues in my project. Perform the following autonomous loop:

Step 1 : Retrieval: Use the Logging MCP to fetch the 5 most recent ERROR entries from the log 'mcp-scenario-logger'.
Step 2 : Iteration: For every unique error found, extract the service and specific error message.
Step 3 : Research: Use the Developer Knowledge MCP to find the official resolution or gcloud command to fix each issue.
Step 4 : Resolution: Consolidate everything into a markdown table with columns: | Service | Error Summary | Recommended Fix |.
```

可能發生的情況

您現在正在即時觀看 **Agentic Workflow**。服務專員會：

- 呼叫 `list_log_entries`，查看我們剛才模擬的「Bad Day」。
- 分析文字，找出 GCS、Cloud Run 和 Secret Manager 失敗的原因。
- 針對每項服務呼叫 `search_documents` 和 `get_document`，找出正確的 IAM 角色或設定修正。
- 顯示類似下表的結構化表格 (建議可能有所不同)：

服務	錯誤摘要	建議修正方式
Cloud Storage	上傳時發生 403 禁止錯誤	將 <code>roles/storage.objectCreator</code> 授予服務帳戶。
Cloud Run	Failed to listen on port 8080	確認應用程式會繫結至 0.0.0.0，並使用 <code>\$PORT</code> 定義的通訊埠。
Secret Manager	缺少版本存取權角色	將 <code>roles/secretmanager.secretAccessor</code> 指派給身分。

步驟 6 · 約 15 分鐘

6. 歷程 C：資料 (Firestore MCP)

在本課程中，您將使用 **Firestore MCP 伺服器**，以自然語言管理 NoSQL 文件資料庫。

Firestore 是彈性且可擴充的資料庫，但管理時通常需要編寫複雜的 SDK 程式碼，或在控制台中操作。有了 MCP，服務專員就能成為**資料庫管理員**，透過即時通訊播送資料、查詢記錄，甚至執行複雜的結構遷移作業。

工作 1：啟用 Firestore 服務

首先，請啟用 Firestore API 和對應的 MCP 端點。

```
# 1. Enable the Firestore API
gcloud services enable firestore.googleapis.com

# 2. Enable the MCP Server interface
gcloud beta services mcp enable firestore.googleapis.com
```

工作 2：指派 Firestore IAM 角色

如要執行查詢，您的身分必須具備基本 MCP 存取權以外的特定權限。

```
# Grant Firestore User role
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="user:$USER_EMAIL" \
  --role="roles/datastore.user"
```

工作 3：建立專屬測試資料庫

為確保實驗安全，我們會建立名為 `mcp-lab-db` 的專屬 Firestore 資料庫。

```
gcloud firestore databases create --database=mcp-lab-db --location=nam5 --type=firestore-native
```

工作 4：設定 Gemini CLI

將 Firestore MCP 伺服器新增至代理。在 `~/.gemini/settings.json` 檔案的 `mcpServers` 區段中新增以下設定。將 `YOUR_PROJECT_ID` 替換為實際專案 ID：

```
"firestore-mcp": {
  "httpClient": "https://firestore.googleapis.com/mcp",
  "authProviderType": "google_credentials",
  "oauth": {
    "scopes": [
      "https://www.googleapis.com/auth/cloud-platform"
    ]
  },
  "timeout": 30000,
  "headers": {
    "x-goog-user-project": "YOUR_PROJECT_ID"
  }
}
```

工作 5：Natural Language DB Ops

啟動 Gemini CLI 並執行一些基本操作，確認連線是否正常。

啟動 Gemini CLI：

```
gemini
```

確認伺服器已準備就緒：輸入 `/mcp list`。畫面上應該會顯示 `firestore-mcp` 和數個工具 (`add_document`, `create_database`, `list_documents`, etc)。

請依序嘗試下列提示：

填充資料：

```
In the 'mcp-lab-db' database, add three documents to a 'products' collection. Include a laptop (stock 5), a mouse (stock 25), and a keyboard (stock 8).
```

驗證：

```
List all documents in the 'products' collection from the 'mcp-lab-db' database.
```

歡迎試試其他提示，透過自然語言管理 Firestore 資料庫和集合。

7. 歷程 D：智慧 (BigQuery 和 Google 地圖)

在本課程中，您將使用 **BigQuery** 和 **Maps Grounding Lite** MCP 伺服器，讓代理程式能夠分析 PB 級資料，並瞭解實體世界。

完成本節後，您的代理程式就能將自然語言轉換為複雜的 SQL 查詢，並提供與情境相關的地理空間建議 (例如旅行時間和天氣)，讓回覆內容更貼近現實。

工作 1：啟用智慧服務

為 BigQuery 和 Google 地圖啟用 API 和 MCP 介面。

```
# 1. Enable product APIs
gcloud services enable bigquery.googleapis.com mapstools.googleapis.com

# 2. Enable MCP Server interfaces
gcloud beta services mcp enable bigquery.googleapis.com
gcloud beta services mcp enable mapstools.googleapis.com
```

工作 2：指派 BigQuery IAM 角色

如要執行查詢，您的身分必須具備基本 MCP 存取權以外的特定權限。

```
# Grant BigQuery Job User and Data Viewer roles
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="user:$USER_EMAIL" \
  --role="roles/bigquery.jobUser"

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="user:$USER_EMAIL" \
  --role="roles/bigquery.dataViewer"
```

工作 3：佈建 Maps API 金鑰

與其他僅仰賴 IAM 的服務不同，**Maps Grounding Lite** 伺服器需要 API 金鑰才能使用配額和計費。

建立金鑰：

```
gcloud alpha services api-keys create --display-name="MCP-Maps-Key"
```

擷取金鑰字串：

```
# Wait a few seconds for the key to propagate, then fetch the string
gcloud alpha services api-keys get-key-string \
  $(gcloud alpha services api-keys list \
    --filter="displayName='MCP-Maps-Key'" \
    --format="value(name)") \
    --format="value(keyString)"
```

複製金鑰字串，以供下一個步驟使用。

工作 4：設定 Gemini CLI

現在，請註冊這兩個伺服器。在 `~/.gemini/settings.json` 檔案的 `mcpServers` 區段中新增下列程式碼片段。請視情況替換 `YOUR_PROJECT_ID` 和 `YOUR_MAPS_API_KEY`。

```
"bigquery-mcp": {
  "httpUrl": "https://bigquery.googleapis.com/mcp",
  "authProviderType": "google_credentials",
  "oauth": {
    "scopes": [
      "https://www.googleapis.com/auth/cloud-platform"
    ]
  },
  "timeout": 30000,
  "headers": {
    "x-goog-user-project": "YOUR_PROJECT_ID"
  }
},
"maps-grounding-lite-mcp": {
  "httpUrl": "https://mapstools.googleapis.com/mcp",
  "headers": {
    "X-Goog-API-Key": "YOUR_MAPS_API_KEY"
  }
}
```

工作 5：實際運用智慧功能

啟動 Gemini CLI 並測試新的「智慧」功能。

```
gemini
```

確認伺服器已準備就緒：輸入 `/mcp list`。畫面上應該會顯示 `bigquery-mcp` 和 `maps-grounding-lite-mcp`，並列出多項工具。。

情境 1：分析引擎 (BigQuery) 即使您不瞭解任何 SQL，也可以要求代理程式查詢公開資料集：

```
Run a query to count the number of penguins on each island in the BigQuery public dataset
ml_datasets.penguins.
```

情境 2：地理空間情境 (地圖) 要求代理規劃真實世界旅程：

```
I am planning a drive from Mumbai to Pune tomorrow morning. Based on current weather and
routing, what should I expect in terms of travel time and what should I carry?
```

報表重點：

- 如果是 BigQuery，代理程式會呼叫 `execute_sql` 來探索結構定義並執行查詢。
- 對於 Google 地圖，這項功能會協調 `lookup_weather` 和 `compute_routes`，為你提供實用且有根據的旅遊計畫。

8. 強化：生產環境安全和 IAM

在最後一個步驟中，您會從使用廣泛的「擁有者」權限，改為採用正式環境等級的縱深防禦模型。

AI 代理本質上就是「實用」。如果您在 UI 層級限制工具，智慧代理程式可能會嘗試執行 Shell 指令來規避限制。如要確實保護基礎架構，您必須使用 **Google Cloud IAM** 建構嚴格的界線。

雙層安全模型

如要執行任何動作，代理程式必須通過兩道閘門：

- 第 1 關 (MCP 關卡)：身分是否具有 `roles/mcp.toolUser`？(使用通訊協定的權限)。
- 第 2 道閘 (服務閘)：身分是否具有特定產品角色 (例如 `roles/datastore.viewer`)？(查看資料的權限)。

工作 1：第 1 層 - 用戶端篩選 (`excludeTools`)

第一層防禦措施是向代理程式隱藏工具，讓代理程式甚至不會「考慮」使用這些工具。

- 在 Cloud Shell 編輯器中開啟 Gemini CLI 設定：

```
cloudshell edit ~/.gemini/settings.json
```

- 找出 `firestore-mcp` 區塊，並新增 `excludeTools` 指令來隱藏破壞性動作：

```
"firestore-mcp": {  
  "httpUrl": "https://firestore.googleapis.com/mcp",  
  "excludeTools": ["delete_database", "update_database", "delete_document"],  
  ...  
}
```

儲存檔案並重新啟動 Gemini CLI。執行 `/mcp list`，並注意這些工具現在已消失。

工作 2：第 2 層 - 基礎架構優勢 (IAM 保全)

用戶端篩選功能是「軟性」防護措施，如果你要求專員「刪除我的 Firestore 資料庫」，但工具已隱藏，系統可能會嘗試執行 `gcloud firestore databases delete`。為避免這種情況，我們使用僅具備必要權限的服務帳戶。

建立「僅供讀取」的服務帳戶：

```
# Create the service account
gcloud iam service-accounts create mcp-reader-sa --display-name="MCP Reader Only"

# Grant ONLY the necessary roles (Gate 1 + Gate 2)
export PROJECT_ID=$(gcloud config get-value project)
SA_EMAIL="mcp-reader-sa@$PROJECT_ID.iam.gserviceaccount.com"

gcloud projects add-iam-policy-binding $PROJECT_ID --member="serviceAccount:$SA_EMAIL" --
role="roles/mcp.toolUser"
gcloud projects add-iam-policy-binding $PROJECT_ID --member="serviceAccount:$SA_EMAIL" --
role="roles/datastore.viewer"
gcloud projects add-iam-policy-binding $PROJECT_ID --member="serviceAccount:$SA_EMAIL" --
role="roles/aiplatform.user"
```

產生及啟用金鑰：

```
gcloud iam service-accounts keys create reader-key.json --iam-account=$SA_EMAIL
export GOOGLE_APPLICATION_CREDENTIALS=$(pwd)/reader-key.json
```

工作 3：「實用代理」彈跳測試

現在，讓我們測試代理程式是否能繞過安全防護。

第一步是啟用服務帳戶，這樣即使代理程式改用 `gcloud` 指令，也會以我們剛建立的服務帳戶身分運作。

啟用服務帳戶：

執行下列指令，並將 [PATH_TO_KEY_FILE] 替換為 JSON 金鑰檔案的實際路徑 (例如 `reader-key.json`)。

```
gcloud auth activate-service-account --key-file=[PATH_TO_KEY_FILE]
```

確認變更：

執行指令後，您可以執行下列指令，確認服務帳戶是否已啟用：

```
gcloud auth list
```

輸出內容會顯示服務帳戶為有效憑證。

啟動 Gemini CLI：

```
gemini
```

輸入以下提示：

```
I want to delete the 'mcp-lab-db' firestore database. If the tool is missing, try using the
gcloud firestore command in the terminal.
```

會發生什麼情況？

1. 代理程式會先嘗試使用 Firestore MCP 伺服器中的 `delete_database` 工具。由於缺少權限，這項作業會失敗。
2. 然後，系統會嘗試「提供協助」，改用 `run_shell_command` 工具執行 `gcloud firestore` 指令。

結果：

指令會失敗並顯示 `Forbidden` 錯誤。由於代理程式是以 `mcp-reader-sa` 身分執行，因此缺少 `datastore.databases.delete` 權限。IAM 是最終的防禦措施。無論代理程式如何嘗試存取資源，Google Cloud API 層級的「Bouncer」都會封鎖要求。

切換回使用者帳戶：

如要切換回使用者帳戶，請輸入下列指令：

```
gcloud config set account YOUR_EMAIL_ADDRESS
```

步驟 9 · 約 1 分鐘

9. 清除所用資源

如要避免產生不必要的費用，請刪除測試資源：

```
# Delete the Firestore database
gcloud firestore databases delete --database=mcp-lab-db

# Remove the service account
gcloud iam service-accounts delete mcp-reader-sa@$PROJECT_ID.iam.gserviceaccount.com
```

10. 結論

恭喜！您已成功瀏覽 Google 代管的 MCP 伺服器完整堆疊。

您首先完成實驗室的「主幹」，也就是建立與 **Cloud Logging** 的基礎連線。接著，您開始進行模組化「冒險」，包括奠定代理程式的知識基礎、自動執行複雜的疑難排解迴圈、在 **Firestore** 中遷移資料，以及從 **BigQuery** 和 **Google 地圖** 中擷取智慧。

最重要的是，您已完成將代理程式錨定在生產安全性的根中。您證明瞭雖然代理程式可能會「好心」過度，但 **Google Cloud IAM** 是最終的保全，可確保自主工作流程一律遵循最小權限原則。

重點回顧

- **代管 = 可擴充**：您透過可串流的 HTTP 連線至基礎架構層級的工具，完全不需要部署伺服器。
 - **建立基準為強制性**：您已使用 **Developer Knowledge MCP** 取代 LLM「猜測」，確保代理程式使用目前有效的指令。
 - **調度是力量**：您已瞭解，代理結合多個 MCP 伺服器來解決單一業務問題時，會產生真正的魔法。
-